

ESP32 Integration Guide for Smart Waste Management Dashboard

Changes Committed

All shadow fixes committed to branch `swm2`.

ESP32 Communication Overview

The dashboard uses **Socket.IO** for real-time communication. Your ESP32 must connect as a Socket.IO client to the backend server (<http://<backend-ip>:3001>).

Commands FROM Dashboard TO ESP32

The backend emits an `esp_control` event for every control action. Your ESP32 should listen for this event:

Command	Payload	Description
<code>power</code>	<code>{ command: 'power', value: true/false }</code>	System ON/OFF - Sent when user presses power button
<code>speed</code>	<code>{ command: 'speed', value: 'slow'/'medium'/'fast' }</code>	Conveyor belt speed
<code>direction</code>	<code>{ command: 'direction', value: 'forward'/'reverse' }</code>	Conveyor belt direction
<code>servo</code>	<code>{ command: 'servo', id: 0-3, value: 0-180 }</code>	Manual servo override

YES, the dashboard can send ON/OFF commands to the Master ESP32!

Data FROM ESP32 TO Dashboard

Your ESP32 should emit these events to update the dashboard:

Event	Payload	Description
<code>item_sorted</code>	<code>{ type: 0/1/2/3 }</code>	Item sorted into bin (0=Wet, 1=Dry, 2=Bio, 3=Hazard)
<code>sensor_data</code>	<code>{ weight: number, volume: number }</code>	Real sensor data for Bio-medical bin

Real vs Simulated Data

☒ REAL Data (requires ESP32)

Data	Source
System ON/OFF	Dashboard → ESP32
Conveyor Speed/Direction	Dashboard → ESP32
Servo Positions	Dashboard → ESP32
Item Sorted Events	ESP32 → Dashboard
Bio-medical Bin Weight/Volume	ESP32 → Dashboard (sensor_data event)
AI Inference Results	Python vision_processor → Dashboard

☐ SIMULATED Data (hardcoded/fake)

Data	Location
Wet/Dry/Hazard bin fill levels	simulationService.js (random increases)
Bin health (battery, signal, motor)	simulationService.js
Revenue calculations	Frontend (\$0.05 × total items)
ECO metrics (trees, CO2)	Frontend (static placeholders)
Predictive charts	Frontend (mock data)
Heatmap data	Frontend (mock data)

 ESP32 Setup Checklist

1. Install Socket.IO Client Library

- Arduino: `ArduinoWebSockets` or `socketio-esp32`
- ESP-IDF: Use WebSocket + JSON parsing

2. Connect to Backend

```
socketIO.begin("YOUR_BACKEND_IP", 3001, "/socket.io/?EIO=4");
```

3. Listen for Commands

```
void onEvent(const char* event, const char* data) {
    if (strcmp(event, "esp_control") == 0) {
        // Parse JSON: { command: "power", value: true }
        // Act accordingly
    }
}
```

4. Send Updates

```
// When item is sorted
socketIO.emit("item_sorted", "{\"type\": 0}"); // 0=Wet

// For Bio bin sensors
socketIO.emit("sensor_data", "{\"weight\": 12.5, \"volume\": 45.2}");
```

Backend Modifications Needed

To make ALL bin data real (not just Bio):

- 1. **Disable simulation for specific bins** in `simulationService.js`
- 2. **Add more `sensor_data` handlers** in `socketService.js` for each bin
- 3. **Create REST endpoints** if you prefer HTTP over WebSocket

Summary

Feature	Status
Dashboard → ESP32 ON/OFF	☑ Already works via <code>esp_control</code> event
Dashboard → ESP32 Speed/Direction	☑ Already works
Dashboard → ESP32 Servos	☑ Already works
ESP32 → Dashboard Item Sorting	☑ Already works via <code>item_sorted</code> event
ESP32 → Dashboard Sensor Data	⚠ Only Bio bin - need to extend for others
Bin fill simulation	📺 Currently simulated, can be made real